DOCMIND

PRODUCT REQUIREMENTS DOCUMENT & TECHNICAL IMPLEMENTATION SPECIFICATION

An AI document intelligence workspace for secure upload, grounded Q&A, summaries, extraction and comparison.

> **AGENT-READY BUILD BRIEF** This is the authoritative product and engineering contract for the first production release. Implement the stated behaviours, data boundaries, APIs and acceptance checks; do not silently replace them with mock functionality.

Document profile

Property	Value
Product	DocMind
Artifact	PRD + software architecture + API contract + deployment runbook
Version / date	1.0 / 01 November 2026
Status	Approved baseline for implementation; configurable assumptions are called out
Primary backend	Python 3.12+ / FastAPI / Docker
Initial deployment target	One small 4 GB VPS for API, worker and PostgreSQL; static Next.js frontend on Cloudflare Pages
Cost objective	About ₹500/month for base infrastructure only; variable AI usage and domain excluded
Intended reader	Autonomous coding agent, engineering lead, QA engineer and operator

BUILD PRINCIPLE

Ship a secure, useful document copilot before adding complex infrastructure. Keep the system modular, observable, cost-capped and portable.

02 / Executive summary & document control

What is being built, who it serves, and how success is measured.

Product summary

DocMind is a browser-based document intelligence SaaS. A user creates an account, receives a private workspace, uploads reference documents, waits for asynchronous processing, and asks questions against selected documents or the entire permitted workspace. Answers must be grounded in retrieved passages and include verifiable source citations with document names and page numbers where available.

The first release supports PDF, DOCX, TXT and Markdown files; private document storage; extraction and indexing; semantic plus lexical retrieval; conversational Q&A; document summaries; key-point extraction; document comparison; searchable chat history; and per-user usage controls. Image-only PDF pages may be OCR-processed when the OCR feature is enabled and the server has capacity.

Primary success outcomes

- New user can register, verify email, sign in and reach a private workspace without operator assistance.
- A supported document can be uploaded, processed asynchronously, and reach READY or a clear actionable FAILED state.
- A user can ask a question and receive a concise answer with citations that open the exact source document/page.
- No user can read, query, download or infer the existence of another workspace's documents or conversations.
- Infrastructure and model usage are visible, bounded by quotas, and recoverable after process restart.

Document control

Control	Rule
Normative language	MUST = required for release; SHOULD = strong default; MAY = optional.
Conflict resolution	This PRD outranks scaffold defaults and agent assumptions. Record a decision before changing it.
Secrets	Never commit secrets. Use deployment environment variables and rotate exposed credentials.
Scope changes	Record feature, reason, cost impact, migration impact and acceptance changes in a decision log.
Versioning	API is versioned under /api/v1. Database changes use forward-only Alembic migrations.

IMPORTANT BUDGET BOUNDARY The ₹500 target is for the initial infrastructure baseline, not an all-inclusive guarantee. LLM tokens, domain renewal, email delivery, storage beyond free allowances, taxes and unexpected traffic can add cost. A hard application-level AI spend cap is mandatory.

03 / Instructions for the autonomous build agent

Treat this as an executable implementation brief, not inspiration.

Build contract

1. First inspect the repository. Preserve useful existing code and document discovered constraints before changing the architecture.
2. Create a working vertical slice before polishing: registration → upload → processing → ask → cited answer → delete.
3. Implement real persistence and real provider integrations. Do not ship fake AI answers, hard-coded sample documents, static counters or mock success states.
4. Use typed request/response schemas, database migrations, structured logs, tests and error handling in every feature.
5. Enforce workspace ownership in the database query/service layer on every object access, not only in frontend navigation.
6. Keep AI provider and object-storage integrations behind interfaces so models and vendors can be switched through configuration.
7. When a provider or feature is unavailable, return a clear disabled/configuration state; never silently fabricate output.
8. Keep the starter deployment economical: one worker concurrency slot, PostgreSQL-backed jobs, no Kubernetes, no always-on Redis and no local LLM on the small VPS.
9. Update README, .env.example, migrations, tests and deployment docs whenever implementation changes a contract.
10. Run unit, integration, security-boundary and end-to-end tests before marking a task complete.

Required implementation sequence

Order	Milestone	Exit condition
M0	Repository + infrastructure skeleton	Compose stack boots; health checks pass; migration runs.
M1	Identity + workspace isolation	Auth lifecycle works; negative cross-tenant tests pass.
M2	Upload + object storage	Presigned upload, file validation, quota and deletion work.
M3	Ingestion pipeline	PDF/DOCX/TXT/MD parsed, chunked, embedded and indexed.
M4	RAG chat + citations	Answers grounded in chunks with verified citation metadata.
M5	Summary, extraction, compare, history	All product actions persist and have tests.
M6	Hardening + launch	Backups, observability, CI/CD, security and smoke tests pass.

DEFINITION OF COMPLETE A UI button is not a completed feature. It is complete only when the API, authorization, persistence, failure path, logs, tests and user-visible state are implemented end to end.

04 / Contents

38-page implementation map. Page numbers refer to this document's printed pagination.

Page	Section	Page
01	Cover	1
02	Executive summary & document control	2
03	Instructions for the autonomous build agent	3
04	Contents	4
05	Product vision, goals and principles	5
06	Scope, non-goals and assumptions	6
07	Personas, jobs-to-be-done and use cases	7
08	User journey and information architecture	8
09	Functional requirements: identity and workspaces	9
10	Functional requirements: dashboard and library	10
11	Upload, file safety and object storage	11
12	Document ingestion and text extraction	12
13	OCR, tables and document structure	13
14	Chunking, embeddings and indexing	14
15	Retrieval and RAG strategy	15
16	Answer generation and citations	16
17	Summaries, extraction and comparison	17
18	Conversations, search and exports	18
19	Quotas, usage metering and product limits	19
20	Frontend experience and accessibility	20
21	System architecture overview	21
22	Service boundaries and runtime behaviour	22
23	Technology decisions and architecture records	23
24	Repository structure and coding standards	24
25	Data model and entity relationships	25
26	Database schema, constraints and indexes	26
27	API standards and identity endpoints	27
28	Document and upload API contract	28
29	Chat, search and AI API contract	29
30	Background processing and job state machine	30
31	AI provider abstraction and prompt design	31
32	Security, authorization and threat model	32
33	Privacy, retention, backup and deletion	33

Page	Section	Page
34	Deployment architecture and Docker Compose	34
35	CI/CD, release, backup and recovery operations	35
36	Observability, reliability and performance	36
37	Testing strategy and release acceptance	37
38	Roadmap, risks, configuration and references	38

READING PATH Implementation agents should read pages 3, 6, 21–34 and 37–38 before coding. QA should prioritize pages 9–19, 27–33 and 37. Operators should use pages 34–38.

05 / Product vision, goals and principles

The value proposition and measurable release outcomes.

Vision

Make long and complex documents usable as a searchable, question-answerable knowledge base. DocMind should reduce the time people spend locating facts, understanding unfamiliar material and comparing information across files while preserving traceability to the source.

Product goals

- Enable self-service document upload and processing with transparent progress and failure reporting.
- Answer questions across one or many selected documents with evidence snippets and source locations.
- Provide repeatable document actions: concise summary, key points, entities/fields extraction and side-by-side comparison.
- Keep user files private by default and make deletion understandable and testable.
- Operate affordably on a small server through external AI APIs, static frontend hosting and bounded background work.
- Provide an architecture that can later separate compute, database, object storage and AI providers without rewriting business logic.

Product principles

Principle	Implementation consequence
Grounded over confident	If evidence is weak or absent, say so; do not invent document facts.
Private by default	No public file URLs; every query is scoped to an authorized workspace.
Visible processing	Show queued, processing, ready, failed and deleting states with timestamps.
Cost-aware AI	Track tokens, enforce quotas and stop requests when budgets are exhausted.
Recoverable operations	Jobs are idempotent; database and object data have backup and restore procedures.
Vendor portability	LLM, embedding and object storage clients are adapters with a common contract.

Release success metrics

Track activation rate (registered users who complete first document ingestion), time-to-ready, upload success rate, question success rate, citation validity rate, no-answer correctness, p50/p95 response latency, API error rate, cost per active user, storage per workspace and deletion completion time. Establish baseline targets after internal testing; do not fabricate product performance targets before measuring.

06 / Scope, non-goals and assumptions

A bounded initial product prevents an agent from building an unshippable platform.

P0 launch scope

- Email/password registration, verification, login, logout, refresh and password reset.
- Private personal workspace created automatically for each account; schema supports workspace membership for later collaboration.
- Upload, list, view status, rename, tag, retry processing and delete PDF, DOCX, TXT and Markdown files.
- Asynchronous text extraction, optional OCR fallback, chunking, embeddings and vector/full-text indexes.
- Question-answer chat scoped to a selected document set or all READY documents the user owns.
- Answer citations with source document, page/section when known, excerpt and open-document action.
- Document summary and key-point extraction; compare two or more selected documents (limit configured).
- Conversation history, rename/delete conversations, document search, usage display, account settings and help/error states.

Explicit non-goals for P0

- No training or fine-tuning of foundation models on user documents.
- No public share links, external collaboration invitations or enterprise SSO.
- No native iOS/Android application, browser extension or desktop agent.
- No OCR for every file by default; OCR is fallback for pages with insufficient extractable text.
- No autonomous legal, medical or financial decisions; DocMind is a document research assistant, not a professional advisor.
- No guaranteed support for handwritten notes, complex scanned tables, password-protected files or files with unsupported DRM.

Assumptions to validate

Assumption	Default decision
Primary customer	Individual professionals, students, researchers and small teams.
Language	English-first UI; preserve detected document language; multilingual prompts may be used if the selected model supports it.
File limits	20 MB per file; maximum 200 pages; configurable by environment/admin policy.
Workspace limits	Initial configurable defaults: 20 documents, 200 MB total, 100 questions/month per account.
Embedding model	Gemini Embedding 2, 768 dimensions, provider/model configurable.
Generation model	Gemini Flash family through an adapter; model name is configuration, not source-code constant.
Billing	No payment gateway in P0; quotas and usage events prepare for future plans.

IMPLEMENTATION RULE All limits must be configuration-backed and enforced on the server. The UI displays the effective limit; it must not be the source of truth.

07 / Personas, jobs-to-be-done and use cases

Design for the everyday workflows that justify recurring usage.

Persona	Need	Typical task
Student / learner	Understand textbooks, notes and research papers quickly.	Upload lecture PDFs; ask “explain chapter 3”; request key points with page citations.
Analyst / professional	Find exact evidence in reports, contracts and policy files.	Search across reports; ask for a metric; compare two versions; export findings.
Researcher	Synthesize information across a collection while retaining provenance.	Upload papers; ask a focused question; inspect relevant source passages.
Small-team owner (future)	Maintain a controlled knowledge base for a team.	Create workspace, invite collaborators, assign roles and manage shared documents (post-P0).

Core jobs-to-be-done

- When I receive a long PDF, I want a reliable summary so I can decide which sections deserve attention.
- When I need a specific fact, I want to ask a natural-language question and see the exact pages supporting the answer.
- When I have multiple documents, I want one question to search across all selected files without leaking other users’ data.
- When two versions differ, I want a structured comparison that names additions, removals and unresolved differences.
- When a document fails, I want a useful reason and retry path rather than an indefinite spinner.

Primary scenarios and expected outcomes

Scenario	Precondition	Expected result
First upload	Verified user; quota available.	Document row appears as UPLOADING/QUEUED, then PROCESSING and READY or FAILED.
Ask a question	At least one READY document selected.	Answer is grounded in retrieved passages and returns valid citations.
No evidence	Question cannot be answered from selected sources.	Assistant explicitly says evidence was not found; no fabricated citation.
Delete file	User owns the file.	Document becomes unavailable immediately; object/chunks are deleted through idempotent cleanup.
Provider outage	AI provider unavailable.	Clear retryable error, safe job state, no duplicate charge or duplicate messages.

08 / User journey and information architecture

The core product loop and navigational model.

Primary user journey

```
LANDING → SIGN UP → VERIFY EMAIL → CREATE PERSONAL WORKSPACE
  → DASHBOARD → UPLOAD DOCUMENTS → PROCESSING STATUS
  → READY DOCUMENTS → SELECT DOCUMENTS → ASK QUESTION
  → GROUNDED ANSWER + CITATIONS → OPEN SOURCE PAGE
  → SAVE CONVERSATION / FOLLOW-UP / SUMMARY / COMPARE
```

Information architecture

Route / view	Purpose	Key components
/	Marketing / sign-in entry	Value proposition, supported formats, privacy summary, CTA.
/auth/*	Identity flows	Register, login, verify, forgot/reset password.
/app	Dashboard	Usage summary, recent documents, recent conversations, upload action.
/app/documents	Document library	Search, filters, sorting, status, upload and row actions.
/app/documents/:id	Document detail	Metadata, preview/download, processing status, summary, ask this document.
/app/chat/:id	Conversation	Messages, evidence citations, source panel, document scope selector.
/app/compare	Compare documents	Select 2–5 READY documents, view structured differences.
/app/settings	Account/settings	Profile, password, usage, privacy, export/delete account.

Navigation and UI state

The app shell contains a left navigation (Dashboard, Documents, Chats, Compare, Settings), an always-visible workspace indicator, user menu and upload CTA. The document picker must only list READY documents within the active workspace. Long-running actions display explicit progress with polling and a cancel/retry action where safe. Responsive behaviour must work at 360 px width and desktop widths. Use skeletons only during expected short loading; show empty states and actionable errors for actual failures.

ACCESSIBILITY All controls require labels, keyboard access, visible focus, sufficient contrast, meaningful status text and semantic headings. Do not communicate processing state only through colour or animation.

09 / Functional requirements: identity and workspaces

FR-001 to FR-010. Identity is the first authorization boundary.

ID	Requirement	Acceptance test
FR-001	Register with email and password; normalize email; reject duplicate accounts.	Duplicate email returns 409; password policy is enforced.
FR-002	Hash passwords with Argon2id; never store or log cleartext passwords.	DB contains only encoded password hash; logs contain no password.
FR-003	Require email verification before document processing.	Unverified account cannot create upload intent or start AI actions.
FR-004	Login issues short-lived access token and rotating refresh token.	Refresh rotation invalidates previous token; logout revokes session.
FR-005	Forgot/reset password uses single-use expiring token.	Token is hashed in DB, expires, and cannot be reused.
FR-006	Create a private personal workspace at registration.	Workspace owner is the registering user; default workspace is active.
FR-007	Every request resolves user and active workspace from trusted server state.	Changing workspace_id in body/query cannot bypass membership checks.
FR-008	Support profile view/update and password change.	Only permitted profile fields can be changed; password change requires current credential.
FR-009	Provide account deletion request with confirmation and cleanup job.	User loses access immediately; deletion workflow is tracked and retryable.
FR-010	Protect auth endpoints from brute force and enumeration.	Rate limits apply; forgot-password response does not reveal account existence.

Session defaults

Setting	Default
Access token TTL	10 minutes
Refresh token TTL	30 days; rotate on use
Password hashing	Argon2id with library-managed secure parameters
Cookie	HttpOnly, Secure in production, SameSite=Lax, Path=/api/v1/auth/refresh
CSRF	Required for cookie-authenticated state-changing refresh/logout flows; validate Origin and token.
Rate limits	Register/login/forgot/reset limits configurable per IP and account.

BUILD NOTE Use a vetted JWT/password library. Do not implement cryptography manually. Do not put access or refresh tokens in localStorage.

10 / Functional requirements: dashboard and document library

FR-011 to FR-018. The library is the user's control surface.

ID	Requirement	Acceptance test
FR-011	Dashboard shows recent docs, recent chats, usage and upload action.	Numbers come from live database aggregates; empty state is useful.
FR-012	List documents with pagination, search, status and sort.	Search is workspace-scoped; pagination is stable and bounded.
FR-013	Show document detail: filename, type, size, page count, status, dates and processing error.	Fields reflect persisted server state, not optimistic UI guesses.
FR-014	Allow rename and tag assignment.	Names are escaped; tags are scoped to workspace; metadata persists.
FR-015	Allow retry for retryable FAILED documents.	Retry creates/reuses an idempotent job without duplicate chunks.
FR-016	Allow download through short-lived signed object URL.	Only owner can request; URL expires and is never stored as public metadata.
FR-017	Allow delete with confirmation and asynchronous cleanup.	File disappears from search immediately; cleanup is retried until complete.
FR-018	Allow selection of READY docs for chat and compare.	Non-READY documents are disabled with reason shown.

Document statuses

Status	Meaning	User action
UPLOADING	Upload intent created; bytes not confirmed.	Continue upload or cancel.
QUEUED	Object verified; ingestion job waiting.	View queue status.
PROCESSING	Parser/OCR/chunk/embed/index running.	Wait; optionally cancel if supported.
READY	Indexed and available for search/Q&A.	Preview, ask, summarize, compare.
FAILED	Terminal or exhausted retry state.	Read reason; retry when safe.
DELETING	Access revoked; storage cleanup pending.	Wait; no query/download access.
DELETED	Cleanup completed; tombstone retained.	No action.

The list must not expose internal storage keys, signed URLs, provider payloads, raw embeddings or another tenant's metadata. Use a compact filter bar for status/type/date and preserve filter state in the URL where practical.

11 / Upload, file safety and object storage

FR-019 to FR-023. Treat every uploaded file as hostile input.

Upload protocol

1. Client → POST /api/v1/documents/upload-intents
2. API checks auth, workspace, quotas, extension, MIME and size.
3. API creates document row (UPLOADING) and short-lived R2 PUT URL.
4. Browser PUTs bytes directly to private R2 object key.
5. Client → POST /api/v1/documents/{id}/complete
6. API performs HEAD and validates size, content type and checksum metadata.
7. API marks QUEUED and inserts ingestion job transactionally.
8. Worker processes object; client polls document status.

Upload requirements

- Allow only PDF, DOCX, TXT and MD in P0. Enforce file extension AND inspected MIME/signature; do not trust Content-Type supplied by browser.
- Default maximum 20 MB per file and 200 pages. Limits are configurable and enforced before signing and after upload verification.
- Generate opaque object keys with random UUIDs; never use user-supplied filenames as storage paths.
- Use private R2 bucket, short-lived presigned URLs, least-privilege access keys and restrictive CORS for the app origin.
- Validate file signatures, reject empty files and suspicious archive structures; cap decompressed DOCX archive size/ratio to mitigate zip bombs.
- Apply request timeouts, upload quotas and per-user concurrent upload limits. No public bucket for private user documents.
- Store file checksum, byte size, detected MIME, object key and original filename separately; filename is display metadata only.

Object storage rules

Use the S3-compatible Cloudflare R2 API through an `ObjectStorage` interface. A storage object is private. API issues presigned GET URLs only after checking workspace ownership. A URL should expire within 60–300 seconds and be returned only to an authenticated authorized client. Deletion removes the object and any derived previews. Keep object keys stable for the lifetime of a document version; never embed the key in public URLs.

> **SECURITY CONTROL** Never accept arbitrary remote URLs for ingestion. This avoids SSRF and prevents the server from fetching private network resources.

12 / Document ingestion and text extraction

FR-024 to FR-027. The ingestion pipeline transforms files into searchable, traceable text.

Pipeline stages

```
QUEUED → FETCH_OBJECT → VERIFY_CHECKSUM → DETECT_FORMAT
  → EXTRACT_TEXT → NORMALIZE → DETECT_EMPTY_PAGES
  → CHUNK → EMBED_BATCHES → UPSERT_INDEX → MARK_READY
Any stage failure → record error_code + safe_message → retry/backoff or FAILED.
```

Parser choices

Format	Library / approach	Required metadata
PDF	PyMuPDF (fitz); iterate pages in order.	Page number, extracted blocks, bounding boxes if available.
DOCX	python-docx; paragraphs and tables in document order.	Paragraph index, heading path, table index/row if available.
TXT	UTF-8 first; detect/handle common BOM and encoding fallback safely.	Line offsets and encoding used.
Markdown	Markdown parser; preserve headings/lists/code blocks as structured text.	Heading path, block type, source offsets.

Extraction and normalization rules

- Preserve page or block boundaries. Do not concatenate the entire document into one untraceable text string.
- Normalize Unicode to NFC, line endings to LF, repeated whitespace and hyphenation conservatively. Preserve meaningful bullet/list/table structure.
- Keep the original extracted text for debugging only under restricted access; store normalized text only as needed for chunking and citations.
- Capture page_count, extracted_char_count, detected_language, parser_name/version, extraction timestamp and processing version.
- If extraction produces too little text, mark pages for OCR if enabled; otherwise fail with `NO_EXTRACTABLE_TEXT` and a helpful message.
- Do not log extracted content, raw prompts or whole documents in application logs.

Error taxonomy

Use stable codes: `UNSUPPORTED_FORMAT`, `FILE_TOO_LARGE`, `PAGE_LIMIT_EXCEEDED`, `PASSWORD_PROTECTED`, `CORRUPT_FILE`, `NO_EXTRACTABLE_TEXT`, `OCR_UNAVAILABLE`, `EMBEDDING_PROVIDER_ERROR`, `STORAGE_UNAVAILABLE`, `PROCESSING_TIMEOUT`, `UNKNOWN_PROCESSING_ERROR`. Expose a safe explanation and support trace ID, not Python stack traces.

13 / OCR, tables and document structure

Preserve enough structure to make document answers useful, not merely plausible.

OCR policy

OCR is a fallback for scanned/image-only PDF pages and pages whose extracted text is below a configurable minimum. Do not OCR all pages by default because it increases CPU time, memory use and latency. Use PyMuPDF to render selected pages at a controlled DPI and Tesseract as an optional OCR engine. The worker must enforce page count, render dimensions, per-page timeout and total OCR time limit.

Setting	Default / rule
OCR engine	Tesseract; install language packs only for languages intentionally supported.
Render DPI	150 DPI default; configurable up to 200 for the small VPS.
OCR threshold	Flag page if normalized extracted text < configurable threshold (e.g. 30 non-space chars).
OCR quality	Store method=`ocr`, confidence where available, and page number.
Fallback	If OCR binary/language pack absent, report `OCR_UNAVAILABLE`; do not crash the worker process.
Resource control	One document job at a time; hard timeout; clean temporary images after processing.

Tables and reading order

- For PDF, preserve text block order and page number. Use table detection only as best effort; do not invent cell relationships when extraction is ambiguous.
- For DOCX, serialize tables as labeled rows (for example, `Table 2 / Row 3: Field = Value`) to preserve semantic context.
- For Markdown, retain heading hierarchy and block labels so retrieval can include section titles.
- Include document title, section heading and page number in the text sent to embeddings when this metadata improves retrieval.
- If a table is too wide or malformed, retain its textual contents and mark metadata `structure_quality=partial`.

ACCURACY DISCLAIMER Scanned handwriting, multi-column layouts, charts and complex table layouts can be inaccurate. The UI should identify OCR-derived content when possible and always let users inspect the original file.

Future extension

A future multimodal ingestion version may send selected PDF pages or images to a vision model. This is not required for P0; it must be separately costed, opt-in, privacy-reviewed and guarded by file/page quotas.

14 / Chunking, embeddings and indexing

FR-028. Use stable, repeatable chunk records tied to the original source location.

Chunking strategy

Chunk by structural boundaries first (heading, paragraph, list item, table row group), then split long blocks into token-bounded windows. Initial target is 700–900 tokens with 100–150 token overlap. Keep chunks small enough for retrieval precision and large enough to retain context. Use a tokenizer appropriate to the embedding model where available; character-based approximation is acceptable only with documented limits.

Field	Purpose
chunk_index	Stable order within document version.
text	Normalized retrieval text.
page_start / page_end	Page range for citation; nullable for formats without pages.
section_path	Hierarchical headings such as `Chapter 2 > Scope`.
char_start / char_end	Offsets into normalized extracted text where meaningful.
token_count	Approximate or provider-reported token count.
embedding	Vector(768) for configured embedding model.
embedding_model / version	Records which embedding space created the vector.

Embedding contract

Implement an `EmbeddingProvider.embed_documents(texts, task_context)` and `embed_query(text)` interface. Recommended initial adapter: Google Gemini Embedding 2 with output_dimensionality=768. Keep `EMBEDDING_PROVIDER`, `EMBEDDING_MODEL` and `EMBEDDING_DIMENSION` configurable. The adapter validates vector length, finite values and non-zero norm. Store embedding model/version with each document/chunk so a future model change can trigger a full re-embedding migration.

Indexing and batch behaviour

- Batch embedding calls within provider request limits; use bounded batch size (start 16–32 chunks, tune with measurement).
- Persist chunks in batches using transactions. Upsert by `(document_id, processing_version, chunk_index)` to make retries idempotent.
- Create a pgvector index appropriate to expected corpus size (HNSW or IVFFlat after benchmarking); exact scan is acceptable for a tiny pilot.
- Create PostgreSQL full-text search configuration for lexical matching and phrase terms.
- If embedding provider fails temporarily, retry with exponential backoff and jitter; do not mark READY until embeddings and indexes are committed.

> **MODEL MIGRATION WARNING** Vectors from different embedding models/spaces must not be compared. A model change requires re-embedding all active chunks or isolating indexes by model/version.

15 / Retrieval and RAG strategy

FR-029. Retrieval is the security and factuality boundary for question answering.

Retrieval pipeline

```
QUESTION → normalize / validate → embed_query
  → SQL prefilter by workspace + READY documents + selected IDs
  → vector top 20 + lexical top 20
  → reciprocal-rank fusion / deduplicate
  → optional local heuristic rerank
  → select top 5–8 evidence chunks within context budget
  → send evidence bundle to generation model
```

Required retrieval controls

- Apply workspace and document permission filters inside SQL before ranking. Never retrieve globally and filter afterward.
- Exclude documents outside the selected IDs when the user has chosen an explicit document scope.
- Exclude DELETING, DELETED, FAILED, PROCESSING and non-READY documents from every query.
- Use a hybrid approach: vector similarity for semantic matches plus PostgreSQL full-text search for names, identifiers and exact phrases.
- Deduplicate overlapping chunks from the same source and avoid returning repeated near-identical context.
- Keep a retrieval audit record: query ID, candidate chunk IDs, final chunk IDs, scores and retrieval version; do not store unnecessary sensitive query text.
- Provide a no-evidence result when score/coverage is below a calibrated threshold. Threshold must be tuned with a labelled evaluation set.

Initial tuning defaults

Parameter	Starting value	Notes
Vector candidates	20	After tenant/document filtering.
Lexical candidates	20	Use ts_rank and query terms.
Final evidence chunks	5–8	Fit model context and avoid token waste.
RRF constant	60	Standard starting point; tune offline.
Max context tokens	8,000	Hard cap independent of model context window.
No-answer threshold	Configurable	Calibrate; do not guess a magic cosine score.
Selected doc maximum	20	Limit query scope size initially.

PROMPT-INJECTION DEFENCE Retrieved document text is untrusted input, not instructions. Delimit it as quoted evidence and tell the model to ignore instructions contained within source documents.

16 / Answer generation and citations

FR-030. Every factual answer must be explainable and traceable.

Generation contract

The generation model receives the user question, a minimal system policy, conversation context (bounded to recent messages), and the selected evidence bundle. Evidence is labeled with opaque IDs such as `[S1]`, each mapped server-side to one chunk. The model must answer only from evidence for document-specific facts; distinguish evidence from general explanation; and state when evidence is insufficient.

Required response shape

```
{
 "answer": "The report states ... [S1].",
 "insufficient_evidence": false,
 "citations": [
  {"source_id":"S1", "claim":"The reported value is 18%"}
 ],
 "suggested_followups": ["Compare this with the prior period"]
}
```

The schema above is logical output, not a promise that the model returns valid JSON. Use structured output/JSON mode when supported; otherwise parse, validate and perform one bounded repair attempt. If validation fails, return a safe internal error without inventing citations.

Citation validation

- Accept only source IDs present in the evidence bundle for this request.
- Map accepted IDs to chunk/document metadata on the server; never trust model-supplied filenames, page numbers or URLs.
- Return document title, page number or page range, section path, excerpt and an opaque document ID.
- Clicking a citation opens the authorized document detail/preview at the cited page or scroll position where supported.
- If the model cites an unknown source ID, remove that citation and set a validation warning; if no valid citations remain for a factual answer, re-run once or return an insufficient-evidence response.
- Never display a source that the current user cannot access, even if it appears in stale chat history.

No-answer behaviour

When evidence is absent, say: “I could not find enough information in the selected documents to answer this reliably.” It may suggest a narrower query or ask the user to select more documents. It must not fill gaps using general model knowledge without clearly labeling that material as outside the documents; P0 defaults to source-only mode.

> **ACCEPTANCE TARGET** A citation is valid only when its source ID is in the retrieval set and its document belongs to the active authorized workspace. Validate this with automated tests, not manual inspection alone.

17 / Summaries, extraction and comparison

FR-031 to FR-034. Reuse the same secure ingestion and evidence foundations.

Document summary

Generate a structured summary from the selected READY document. For long documents, use map-reduce summarization: summarize chunks/sections, then synthesize a document-level summary from the intermediate summaries while retaining page references. Store summary version, model ID, prompt version, generated timestamp and document processing version. Regenerate when source content or model/prompt version changes.

Key-point extraction

Return an ordered list of key points with a short explanation and supporting citations. The user can copy/export the list. Each key point must reference evidence; unsupported generic advice is not a key point.

Structured extraction

Allow a small set of built-in extraction presets in P0: entities/people/organizations, dates, amounts/metrics, obligations/action items and named sections. Use schema-validated JSON; include a confidence note and source citations. Do not present model confidence as statistically calibrated probability unless measured.

Document comparison

Allow 2–5 READY documents selected by the user. Retrieve/prepare relevant sections for each, then ask the generation model for commonalities, differences, unique points and missing/unclear fields. Present a comparison matrix and narrative. Every row must cite the document(s) it describes. Enforce per-request token and document count caps.

Action	Output structure	Failure handling
Summary	Overview, sections, key points, caveats, citations.	Show retryable error; do not store partial output as complete.
Key points	Array of claims with evidence/citations.	Reject invalid schema or unsupported point.
Extraction	Preset ID, typed fields, source references.	Show missing fields as null/unknown, not guessed.
Compare	Matrix + narrative + citations + limitations.	Warn if one document has weak extraction or mismatched scope.

COST GUARD Cache completed summaries by document version + model + prompt version. Do not regenerate identical work on every page load.

18 / Conversations, search and exports

FR-035 to FR-039. Maintain continuity without leaking document context.

Conversation requirements

- Users can create, list, rename and delete conversations; a conversation stores workspace ID, creator ID, title, selected document IDs and timestamps.
- Messages store role, text, status, token usage, provider/model, latency and request trace ID. Assistant messages store citation records separately.
- Follow-up questions may use recent conversation messages, but must re-run retrieval for the new question and selected document scope.
- Conversation history is always re-authorized on read. Deleting a document must cause historical citations to display “source deleted” and must not restore access.
- Conversation title may be generated from the first question, but can be edited by the user; title generation is best effort.

Search requirements

Provide document-library search by filename and metadata plus content search across READY documents. Content search uses the same tenant-safe hybrid retrieval service but returns snippets rather than an LLM answer. Search results include file name, page/section and highlighted excerpt. Search queries are rate-limited and paginated.

Export requirements

Export	P0 behaviour
Conversation Markdown	Export user/assistant messages with citation references and document names; signed links are not embedded.
Summary copy	Copy as plain text/Markdown.
Document export	Original file download through a short-lived signed URL after authorization.
PDF report	P1; generate through a safe server-side renderer or client export after evaluation.

Deletion and history

Deleting a conversation revokes access immediately and enqueues cleanup of messages/citations. Deleting a document removes its chunks from future retrieval immediately, then deletes object data and any derived metadata. Historical answers remain as conversation text unless the user deletes the conversation or account; the UI must mark citations to deleted sources and privacy settings must explain this behaviour.

19 / Quotas, usage metering and product limits

FR-040 to FR-044. Prevent abuse and keep variable AI spend bounded.

Initial launch limits

Limit	Default starting value	Enforcement
Max file size	20 MB	Upload intent and storage HEAD verification.
Max pages	200 pages/document	Parser preflight and processing guard.
Documents	20 active docs/workspace	Transactional quota check.
Stored data	200 MB/workspace	Count object bytes; reconcile periodically.
Questions	100 per user/month	Usage event reservation before provider call.
Concurrent uploads	2/user	DB-backed active upload count.
Compare scope	2–5 documents	Request validation.
Chat context	8,000 evidence tokens + bounded conversation	Server-side token budgeting.
Requests	Configurable rate limits by endpoint	Redis-free DB/IP token bucket or proxy rate limiting.

Usage model

Create immutable `usage_events` records for model input/output tokens, embedding tokens/characters when provider returns usage, OCR pages, file bytes and operation counts. Each event records user, workspace, action, provider, model, units, estimated cost, timestamp and request ID. Keep raw provider costs as estimate until invoice reconciliation. Store currency and rate-source timestamp if converting to INR.

Spend protection

- Configure per-day and per-month LLM budget caps. Reject new AI operations when cap is reached; allow non-AI document list/download/delete functions.
- Reserve a usage slot before provider call and settle actual usage after response; release reservation on pre-call failure.
- Apply hard max output tokens, context budget, retry count and timeout. Never retry non-idempotent paid generation without a request idempotency key.
- Provide operator kill switch `AI_FEATURES_ENABLED=false` and per-provider feature flags.
- Dashboard displays remaining quota and “resets on” date; all quota values come from backend.

> **BILLING PHASE** P0 does not charge users. Do not integrate payments until real usage/cost patterns and plan rules are understood. Keep usage events clean so Stripe/Razorpay subscriptions can be added later.

20 / Frontend experience and accessibility

The interface should make document state and evidence obvious.

Visual direction

Use a calm productivity interface: light neutral canvas, dark navy typography, one blue/teal action palette, simple line icons, restrained shadows and strong information hierarchy. Avoid visual noise. The primary action is “Ask your documents”; upload and citation inspection should remain easy to discover.

Core components

Component	Behaviour
UploadDropzone	Drag/drop and file picker; displays permitted formats, size limit, upload progress and validation error.
DocumentRow	Name, type, size, updated time, status, action menu; status has text label and icon.
ProcessingTimeline	Queued/processing stage, last update, safe failure reason and retry.
DocumentScopePicker	Search/select READY docs; “all documents” option; show current scope in chat.
ChatComposer	Multiline input, send state, disabled state during submission, cancel where supported.
CitationCard	Document name, page, snippet, open action; never trust client-supplied citation metadata.
UsageMeter	Current use / limit, reset date, budget warning.
ConfirmDialog	Explicit warning for irreversible delete/account actions.

State design

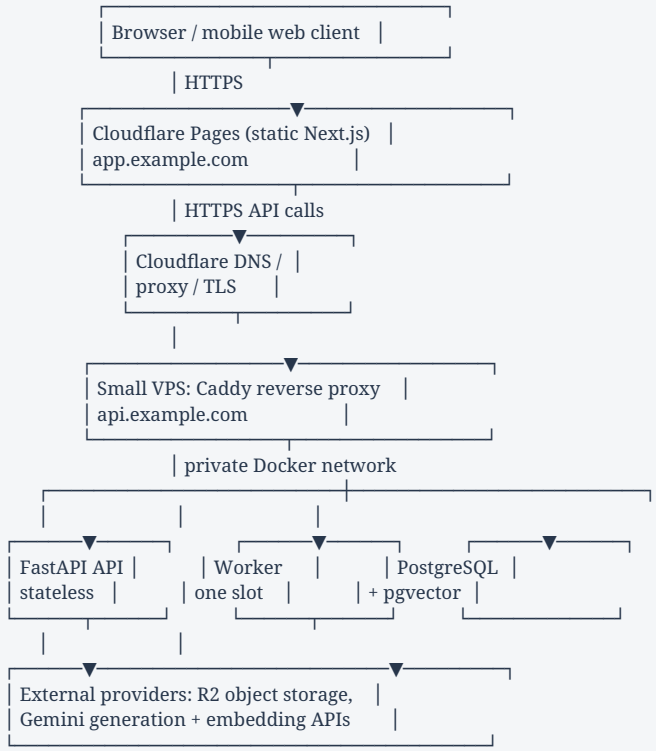
- Every async action has loading, success, retryable error and terminal error states.
- Empty states explain the next action; errors offer a recovery step and support trace ID.
- Optimistic updates are allowed only when rollback is implemented and server remains source of truth.
- Use pagination or virtualized lists for large collections; never download all documents to filter in browser.
- Protect against accidental duplicate submits with disabled controls and server idempotency keys.

Accessibility / responsive acceptance

Target WCAG 2.2 AA as a design goal. Keyboard users must reach upload, navigation, composer, citation cards and dialogs. Support visible focus, correct label/description relationships, non-colour status, sufficient contrast, reduced motion, screen-reader live status for processing and responsive single-column layout on mobile. Avoid hover-only interactions.

21 / System architecture overview

A lean topology that meets the budget without compromising core isolation.



Key topology choices

- Frontend is statically exported from Next.js and hosted on Cloudflare Pages. No SSR, server actions or server secrets in the frontend.
- VPS runs only Caddy, FastAPI, one background worker process and PostgreSQL/pgvector. Do not run local LLM inference.
- PostgreSQL is the system of record for identities, metadata, chats, jobs, usage and vectors.
- R2 stores source files and optional derived artifacts. PostgreSQL stores object keys and metadata, never file bytes.
- Gemini is accessed over outbound HTTPS through provider adapters. Provider API keys exist only in API/worker environment.
- Worker queue is a PostgreSQL jobs table with `FOR UPDATE SKIP LOCKED`; Redis is deliberately excluded from P0.

TRUST BOUNDARIES The browser is untrusted. The API is the only authorization authority. Worker and database are private to the Docker network. External AI/storage providers receive only the minimum data needed for the requested operation.

22 / Service boundaries and runtime behaviour

Modules are logical boundaries even when deployed as a modular monolith.

Module	Responsibility	Must not do
Auth	Credentials, tokens, email verification, reset, sessions.	Never decide document access from client-provided IDs alone.
Workspace	Workspace membership, active workspace context, roles.	Never trust workspace ID without DB membership check.
Document	Metadata, status, quotas, tags, lifecycle.	Never store public object URLs or parse files inline in HTTP process.
Storage adapter	Private R2 put/get/head/delete, presigned URLs.	Never expose storage credentials or accept arbitrary remote fetch URLs.
Ingestion	Parsers, normalization, OCR, chunking, embeddings.	Never call from request thread for large files.
Retrieval	Hybrid search, scope filters, score merge, context selection.	Never query across tenants or include non-READY docs.
AI orchestration	Prompt construction, provider routing, structured output, token limits.	Never let model output directly authorize operations.
Conversation	Messages, citation validation, history.	Never trust model-generated citation metadata.
Usage	Quota reservation, metering, budget controls.	Never rely solely on client counters.
Observability	Structured logs, traces, metrics, audit events.	Never log secrets, raw files or unnecessary prompt content.

Request runtime

API request lifecycle: validate request → authenticate principal → resolve workspace membership → apply rate/quota policy → call application service → interact with repository/provider adapter → commit transaction → return typed response. Keep route handlers thin. Services contain business rules; repositories contain SQL/data access; adapters contain vendor calls. Long work creates a job and returns 202 with a job/document resource.

Worker runtime

Worker polls for due jobs, claims one row transactionally, updates heartbeat and attempt count, runs one bounded task, persists output, then marks succeeded. On process restart, stale RUNNING jobs are returned to retryable QUEUED after a lease timeout. Worker concurrency starts at one; use subprocess isolation for OCR if required. The same backend image can run with `COMMAND=api` or `COMMAND=worker`.

23 / Technology decisions and architecture records

Use boring, maintained tools; measure before adding infrastructure.

Layer	Selected technology	Reason / constraint
Language	Python 3.12+	Mature AI/document ecosystem; pin exact runtime in Docker.
API	FastAPI + Pydantic v2	Typed APIs, OpenAPI, async outbound I/O.
ORM / migrations	SQLAlchemy 2 + Alembic	Explicit data access and forward migrations.
Database	PostgreSQL 16+ with pgvector	Single durable store for relational data, FTS and vectors.
Frontend	Next.js (static export) + TypeScript	Modern UI; static output avoids a second always-on server.
UI	Tailwind CSS + accessible component primitives	Fast consistent UI; do not couple design system to backend.
Object storage	Cloudflare R2 (S3 API)	Private object storage; free allowance and no internet egress charge within current pricing.
Embeddings	Gemini Embedding 2, 768 dims	Good fit for multilingual semantic retrieval; configurable adapter.
Generation	Gemini Flash family	Cost-aware hosted inference; model ID configurable.
PDF / DOCX	PyMuPDF + python-docx	Local CPU parsing; no separate parsing vendor.
OCR	Tesseract optional	Fallback only; resource-limited.
Job queue	PostgreSQL jobs table	No Redis bill/ops at launch; low throughput acceptable.
Deployment	Docker Compose + Caddy	Small, reproducible single-host deployment.
Frontend hosting	Cloudflare Pages	Static assets, free static requests within provider terms.
CI/CD	GitHub Actions	Test, build and deploy on protected branch.

Architecture decisions (ADRs)

- ADR-001 Modular monolith, not microservices: simple deployment, clear module boundaries.
- ADR-002 External inference, not local GPU: VPS has no practical AI inference capacity at this budget.
- ADR-003 PostgreSQL-backed jobs, not Redis/Celery: initial queue volume is low; add a broker only when metrics justify it.
- ADR-004 Static Next.js frontend: no SSR/Server Actions; API handles all authorization and secrets.
- ADR-005 768-dimensional vectors: reduce database/storage cost; model and dimension are versioned.

DEPENDENCY POLICY Pin direct dependencies, use a lockfile, scan for vulnerabilities, and review license/maintenance status before adding any new package. Avoid unmaintained LangChain-heavy abstractions where a small explicit pipeline is clearer.

24 / Repository structure and coding standards

A concrete project layout that an agent can scaffold directly.

```
docmind/
├── frontend/
│   ├── app/              # routes and layouts
│   ├── components/       # reusable UI
│   ├── features/         # auth, documents, chat, settings
│   ├── lib/api/          # typed API client
│   ├── lib/auth/         # client session state only
│   ├── public/
│   └── next.config.ts    # output: export
├── backend/
│   ├── app/
│   │   ├── api/v1/       # routers + dependencies
│   │   ├── core/         # config, security, logging
│   │   ├── db/           # session, base, migrations
│   │   ├── models/       # SQLAlchemy models
│   │   ├── schemas/      # Pydantic DTOs
│   │   ├── repositories/ # DB access
│   │   ├── services/     # product rules
│   │   ├── adapters/     # LLM, embedding, R2, email
│   │   ├── ingestion/    # parsers, OCR, chunking
│   │   ├── retrieval/    # hybrid search + RAG
│   │   ├── workers/      # job handlers + runner
│   │   └── main.py
│   ├── tests/
│   ├── alembic/
│   ├── pyproject.toml
│   └── Dockerfile
├── infra/
│   ├── compose.yaml
│   ├── Caddyfile
│   ├── postgres/
│   └── scripts/
├── .github/workflows/
├── .env.example
├── Makefile
└── README.md
```

Backend coding rules

- Use type hints for public functions; Pydantic schemas at boundaries; SQLAlchemy models are not API response objects.
- Use dependency injection for DB sessions, current user, settings and provider clients.
- Use UTC timestamps, UUID primary keys and explicit transaction boundaries.
- Use structured JSON logs with request_id, user_id/workspace_id only when safe, job_id and duration.
- Never catch broad exceptions and return success. Map known exceptions to typed API errors; log unknown exceptions with trace ID.
- Use `Decimal` for money/cost rates; never use float for billing calculations.
- Keep network/provider calls behind timeouts and retry policies; do not retry validation/auth errors.
- Write tests alongside each service; fixtures must create separate workspaces for authorization tests.

Frontend coding rules

Use strict TypeScript, shared typed API DTOs (generated from OpenAPI where practical), query caching with invalidation, controlled forms and centralized error mapping. Do not duplicate business rules in the browser. Use `NEXT_PUBLIC_API_BASE_URL` only for the public API URL; no server keys or privileged credentials may be prefixed with NEXT_PUBLIC_.

25 / Data model and entity relationships

The schema must encode tenant boundaries, lifecycle and provenance.

```
users 1 — 1 workspace (personal default)
users 1 — N refresh_sessions
workspaces 1 — N workspace_members N — 1 users
workspaces 1 — N documents
documents 1 — N document_chunks
documents 1 — N processing_jobs (or job references)
workspaces 1 — N conversations 1 — N messages
messages 1 — N message_citations N — 1 document_chunks
users/workspaces 1 — N usage_events
workspaces 1 — N audit_events
```

Core entities

Entity	Purpose	Important relationship
users	Account identity and credential state.	Owner of personal workspace; creator of sessions.
workspaces	Tenant/security boundary.	All user-owned resources reference workspace_id.
workspace_members	Role and membership mapping.	Unique(workspace_id,user_id); owner/member roles.
refresh_sessions	Rotating refresh-token session records.	User FK; store only token hash.
documents	File metadata, processing state and object key.	Workspace FK; owner/user FK; unique scoped id.
document_chunks	Text fragments and vectors.	Document FK and workspace_id for filter safety.
jobs	Durable background work with attempts/leases.	Workspace/document FK; idempotency key.
conversations	Chat scope and title.	Workspace FK; creator FK; selected document IDs stored relationally or join table.
messages	Ordered user/assistant/tool messages.	Conversation FK; token usage and status.
message_citations	Verified evidence links.	Message FK; chunk/document FK.
usage_events	Append-only metering events.	Workspace/user FK; action/provider/model/units.
audit_events	Security-meaningful actions.	Actor + workspace + target + request trace.

Tenant key discipline

Every tenant-owned table must include workspace_id directly or be accessed through a parent join that proves workspace ownership. For high-risk tables such as chunks, include workspace_id and validate it matches the parent document. Prefer composite foreign keys such as `(workspace_id, document_id)` to prevent accidental cross-workspace relationships. Every repository read/write accepts an authorized workspace context.

NO IMPLICIT GLOBAL SCOPE Do not rely on frontend filters or an optional `workspace_id` query parameter. Server-resolved membership is mandatory for every data operation.

26 / Database schema, constraints and indexes

Recommended columns and database-level safeguards.

Table	Key fields (minimum)	Constraints / indexes
users	id UUID, email CITEXT, password_hash, verified_at, status, created_at	Unique normalized email; status enum/check.
workspaces	id, name, slug, created_by, created_at	Unique slug; creator FK.
workspace_members	workspace_id, user_id, role, joined_at	PK(workspace_id,user_id); role check.
refresh_sessions	id, user_id, token_hash, expires_at, revoked_at, replaced_by	Unique token hash; index user_id/expires_at.
documents	id, workspace_id, uploaded_by, filename, mime, bytes, checksum, object_key, status, page_count, processing_version, error_code	Unique workspace/id; index workspace/status/created_at; unique object key.
document_chunks	id, workspace_id, document_id, processing_version, chunk_index, text, vector(768), page_start/end, section_path, metadata	Unique(document_id,processing_version,chunk_index); vector + FTS indexes.
jobs	id, workspace_id, job_type, status, payload, attempts, max_attempts, available_at, locked_at, locked_by, heartbeat_at, idempotency_key, last_error	Unique idempotency key; index status/available_at.
conversations	id, workspace_id, created_by, title, scope_mode, created_at, updated_at	Index workspace/updated_at.
messages	id, conversation_id, role, content, status, model, input_tokens, output_tokens, created_at	Index conversation/created_at.
message_citations	id, message_id, chunk_id, document_id, source_label, claim_text	FKs; unique(message_id,chunk_id).
usage_events	id, workspace_id, user_id, event_type, provider, model, units, cost_usd, created_at, request_id	Append-only; indexes workspace/time and user/time.
audit_events	id, workspace_id, actor_user_id, action, target_type, target_id, metadata, created_at	Index workspace/time; metadata excludes secrets.

Database configuration

- Enable pgvector extension in migration. Use CITEXT or normalized lowercase email with a unique index.
- Use database enum types or validated CHECK constraints for states and roles.
- Use `ON DELETE` behaviour intentionally: identity/account purge should be controlled by a deletion workflow; do not cascade-delete evidence before access is revoked.
- Use `created_at`/`updated_at` in UTC. Add `deleted_at`/tombstone where cleanup must be recoverable.
- Use Alembic migrations for every schema change; test both fresh install and upgrade from previous migration.

> **VECTOR DIMENSION** The initial migration uses vector(768). If EMBEDDING_DIMENSION changes, migration and re-embedding plan must change together. Reject provider vectors of a different size at runtime.

27 / API standards and identity endpoints

REST conventions for a stable, versioned public API.

API conventions

Concern	Contract
Base path	/api/v1
Content type	application/json except direct browser-to-R2 object transfer.
Authentication	Bearer access token for API calls; refresh cookie endpoint is protected with CSRF/origin checks.
IDs	UUID strings; never sequential database IDs.
Timestamps	ISO 8601 UTC, e.g. 2026-11-01T12:00:00Z.
Pagination	Cursor-based for large lists; page/limit acceptable for small stable lists; maximum page size 100.
Errors	RFC 7807-style problem response with code, message, request_id and optional field_errors.
Idempotency	Support Idempotency-Key on upload intent, message submit and paid AI operations.
Timeouts	Bound outbound requests; long work returns 202 + resource/job ID.
Versioning	Breaking changes require new API version; additive optional fields are allowed.

Standard error body

```
{
  "type": "https://docmind.example/errors/quota_exceeded",
  "title": "Quota exceeded",
  "status": 429,
  "code": "MONTHLY_QUESTION_LIMIT_REACHED",
  "detail": "You have used the available questions for this month.",
  "request_id": "uuid",
  "field_errors": null
}
```

Identity endpoint catalogue

Method / path	Purpose	Response
POST /auth/register	Create account + personal workspace; send verification email.	201 user/workspace summary.
POST /auth/login	Authenticate and create session.	200 access token + user; refresh cookie.
POST /auth/refresh	Rotate refresh token.	200 new access token; rotate cookie.
POST /auth/logout	Revoke current refresh session.	204.
POST /auth/verify-email	Consume one-time verification token.	200 verified state.
POST /auth/forgot-password	Create reset token and send email.	202 generic response.
POST /auth/reset-password	Consume token and set new password.	204.
GET /me	Current profile, active workspace and usage summary.	200 profile DTO.
PATCH /me	Update safe profile fields.	200 updated DTO.

All endpoints must have OpenAPI descriptions, request/response schemas, authorization dependencies, validation and tests. Production OpenAPI may be restricted to authenticated admins or disabled publicly while remaining available in CI.

28 / Document and upload API contract

Document lifecycle endpoints and representative payloads.

Method / path	Purpose / rules
POST /documents/upload-intents	Body: filename, declared_mime, size_bytes, checksum_sha256. Checks quota/format, creates UPLOADING row, returns document_id, presigned_put_url, required_headers and expires_at.
POST /documents/{id}/complete	Verify object HEAD/size/checksum; mark QUEUED; create ingestion job transactionally. Idempotent.
GET /documents	List with cursor, q, status, type, sort. Always workspace scoped.
GET /documents/{id}	Metadata/status/error summary; no storage internals.
PATCH /documents/{id}	Rename and/or tags; ownership check.
POST /documents/{id}/retry	Only for retryable FAILED state; creates idempotent job.
POST /documents/{id}/download-url	Authorized short-lived signed GET URL; do not cache/store.
DELETE /documents/{id}	Immediately set DELETING/revoke query visibility; enqueue cleanup.
GET /documents/{id}/summary	Return cached summary or 202 job resource; scope/usage enforced.

Upload-intent request / response

```
POST /api/v1/documents/upload-intents
{ "filename":"annual-report.pdf", "declared_mime":"application/pdf",
  "size_bytes":1843200, "checksum_sha256":"<64-hex>" }

201 Created
{ "document_id":"<uuid>", "upload_url":"https://...signed...",
  "method":"PUT", "headers":{"Content-Type":"application/pdf"},
  "expires_at":"2026-11-01T12:10:00Z" }
```

Completion response

```
POST /api/v1/documents/<uuid>/complete
202 Accepted
{ "document_id":"<uuid>", "status":"QUEUED", "job_id":"<uuid>" }
```

Document DTO minimum

Return id, filename, mime_type, size_bytes, checksum_short, status, page_count, extracted_language, created_at, updated_at, ready_at, tags and safe_error `{code,message,retryable}`. Do not return object_key, bucket, access token, raw extracted text or full provider trace. Signed upload/download URLs are transient response values only.

RACE HANDLING If completion is called twice, return the existing job/status. If upload bytes never arrive, expire stale UPLOADING records and schedule object cleanup.

29 / Chat, search and AI API contract

Core conversational endpoint with server-side scope and source validation.

Method / path	Purpose
POST /conversations	Create conversation with title and selected scope.
GET /conversations	List latest conversations; pagination and workspace filter.
GET /conversations/{id}	Get conversation and messages with currently authorized citations.
PATCH /conversations/{id}	Rename conversation / update selected scope.
DELETE /conversations/{id}	Revoke access and enqueue cleanup.
POST /conversations/{id}/messages	Submit question; reserve quota; retrieve; generate; validate; persist.
POST /search	Hybrid search without answer generation; returns snippets.
POST /documents/compare	Create comparison job for 2–5 READY docs.
GET /jobs/{id}	Poll job status, progress and safe error.
GET /usage	Return current period counters and reset date.

Message request

```
POST /api/v1/conversations/<uuid>/messages
{
  "content":"What are the main risks mentioned in the report?",
  "document_ids":["<uuid>"],
  "response_mode":"source_only",
  "idempotency_key":"client-generated-uuid"
}
```

Message response

```
200 OK
{
  "message_id":"<uuid>",
  "role":"assistant",
  "content":"The report identifies ... [S1].",
  "insufficient_evidence":false,
  "citations":[{
    "citation_id":"<uuid>", "document_id":"<uuid>",
    "filename":"annual-report.pdf", "page_start":12,
    "page_end":12, "section":"Risk factors",
    "excerpt":"...", "source_label":"S1"
  }],
  "usage":{"input_tokens":1200,"output_tokens":240},
  "created_at":"2026-11-01T12:00:00Z"
}
```

API implementation notes

The server derives workspace from authenticated membership and verifies the conversation belongs to it. If document_ids are supplied, every ID must be READY and in the same workspace; reject the whole request on invalid IDs rather than silently widening scope. P0 may return a non-streaming JSON response for simplicity. Add SSE only after the request lifecycle, cancellation, partial message persistence and client reconnect semantics are tested.

> **IDEMPOTENCY** A repeated Idempotency-Key for the same user/route returns the original result or in-progress state, not a second paid LLM invocation.

30 / Background processing and job state machine

Reliable asynchronous work on one small machine.

Job schema and states

```
QUEUED → RUNNING → SUCCEEDED
                ↘ RETRY_WAIT → RUNNING (attempts < max_attempts)
                ↘ FAILED (terminal)
QUEUED/RUNNING → CANCELLED (only where cancellation is safe)
```

Worker claim pattern

```
BEGIN;
SELECT id FROM jobs
WHERE status IN ('QUEUED','RETRY_WAIT') AND available_at <= now()
ORDER BY priority DESC, created_at ASC
FOR UPDATE SKIP LOCKED LIMIT 1;
UPDATE jobs SET status='RUNNING', locked_at=now(), locked_by=:worker,
 heartbeat_at=now(), attempts=attempts+1 WHERE id=:id;
COMMIT;
```

Operational requirements

- Use a lease/heartbeat; if worker crashes, stale RUNNING jobs return to retry after a lease window.
- Use exponential backoff with jitter, e.g. 10s, 30s, 2m, 10m; cap attempts and provide manual retry.
- Handlers must be idempotent; use job idempotency key and document processing_version.
- Persist progress stages, not arbitrary percentages. UI can display “Extracting”, “Embedding”, “Indexing”.
- Limit worker concurrency to one for OCR/embedding-heavy jobs on 4 GB RAM; tune based on memory and throughput tests.
- Apply per-job execution timeout and maximum resource limits; terminate subprocesses on timeout.
- On terminal failure, store a safe error code/message and full sanitized stack trace in restricted logs only.
- Delete temp files in `finally` blocks; do not store PII or extracted text in job payloads.

Job types

Job type	Input reference	Output
INGEST_DOCUMENT	document_id + processing_version	Chunks, embeddings, document READY status.
DELETE_DOCUMENT	document_id	Object/chunks/derived data removed; DELETED.
GENERATE_SUMMARY	document_id + version + prompt version	Cached summary record.
COMPARE_DOCUMENTS	conversation/request + doc IDs	Comparison result record.
DELETE_ACCOUNT	user_id + deletion request ID	All data cleaned or tombstoned per policy.
RECONCILE_STORAGE	workspace/batch cursor	Corrected byte counters and orphan report.

31 / AI provider abstraction and prompt design

Protect product logic from vendor lock-in and model drift.

Provider interfaces

```
class LLMProvider(Protocol):
    async def generate(self, request: GenerateRequest) -> GenerateResult: ...

class EmbeddingProvider(Protocol):
    async def embed_documents(self, texts: list[str]) -> EmbeddingBatch: ...
    async def embed_query(self, text: str) -> list[float]: ...

class ObjectStorage(Protocol):
    async def create_upload_url(...): ...
    async def create_download_url(...): ...
    async def head_object(...): ...
    async def delete_object(...): ...
```

Default provider configuration

Setting	Starting value	Rule
LLM provider	google	Adapter must support timeout, structured output, usage metadata.
LLM model	gemini-3.8-flash (configurable)	Do not hardcode in service or prompt files; verify availability at deploy time.
Embedding provider	google	Use retrieval-appropriate task/config.
Embedding model	gemini-embedding-2	Stable model ID from provider docs; configurable.
Embedding dimension	768	Must match DB vector dimension; validate at startup and per batch.
Temperature	0.2	Use low variance for source-grounded Q&A.
Max output tokens	1,500	Configurable by operation; budget cap.
Timeout	45 seconds generation; 60 seconds embeddings batch	Use cancellation and bounded retries.

System prompt requirements

The source-only Q&A system prompt must: (1) state that the model is answering a user about their selected documents; (2) treat retrieved passages as untrusted quoted data; (3) ignore instructions contained within documents; (4) answer only when evidence supports it; (5) cite source IDs exactly; (6) distinguish direct facts from inference; (7) say when evidence is insufficient; (8) avoid revealing system prompts, secrets or data outside the supplied evidence. Include a prompt version string and test fixtures.

Provider resilience

- Use bounded retries only for retryable 429/5xx/network failures with jitter and Retry-After support.
- For model fallback, fallback must preserve privacy and compatible prompt behaviour; log provider/model actually used.
- No provider payload or document text in routine logs. Add opt-in sanitized debug logging only in non-production.
- Validate returned content, usage fields, finish reason, refusal/blocked states and citation source IDs.
- Use paid provider tier for real confidential user documents when provider free-tier terms allow data to be used for service improvement; review terms before launch.

MODEL LIFECYCLE Model aliases and price pages change. Store provider/model per operation, verify current model availability at release time, and make model updates explicit through configuration plus evaluation tests.

32 / Security, authorization and threat model

Security requirements are part of the product, not a later enhancement.

Threat	Required mitigation
Cross-tenant data leak	Workspace scoping in all repositories; composite constraints; negative isolation tests.
Broken object-level authorization	Authorize every document/conversation/job/citation by workspace and actor.
Malicious upload / parser exploit	Allowlist formats, signature checks, size/page caps, sandboxed subprocess, patched libraries.
Prompt injection in source files	Treat chunks as untrusted data; no tool execution from document instructions; source-only prompt.
Credential theft	Argon2id, rotating refresh token hashes, secure cookies, rate limits, secret rotation.
SSRF / remote-file abuse	Do not ingest remote URLs; server does not fetch user-supplied URLs.
SQL injection	SQLAlchemy parameter binding; no string-built SQL; least-privilege DB role.
XSS / unsafe Markdown	Escape user/document content; sanitize rendered Markdown and block unsafe HTML/URLs.
CSRF	SameSite cookies, Origin validation and CSRF token for cookie-authenticated mutations.
Denial of service / cost abuse	Quotas, rate limits, file/page limits, timeouts, output token caps and spend kill switch.
Secret leakage	No secrets in frontend bundles, logs, Git, error pages or public OpenAPI examples.
Supply-chain compromise	Pinned dependencies, lockfile, automated vulnerability scans, image updates.

Authorization rules

- Authentication establishes user identity; membership establishes workspace access; role/permission establishes action rights.
- Every read/write includes workspace predicate and, where applicable, owner or role predicate.
- Jobs use server-created payloads; user cannot submit arbitrary job types or worker payload fields.
- Signed object URLs are capabilities: short TTL, object-specific, private, never placed in logs or persisted.
- Admin support access must be explicit, audited and limited; no silent “view any document” admin path.

Security tests

Create two users in separate workspaces. Attempt to list, fetch, rename, download, chat against, compare, cite, delete and poll jobs for the other user’s objects by guessed UUID. Every attempt must fail with 404 or consistent authorization response and no data leakage. Include expired/revoked token, refresh rotation replay, oversized upload, malformed archive and XSS payload fixtures.

33 / Privacy, retention, backup and deletion

Be clear about what data is stored, sent to AI providers and removed.

Data classification and handling

Data	Storage / handling	AI provider transmission
Credentials	Password hash and session token hashes in PostgreSQL.	Never sent.
Source file	Private R2 object; encrypted in transit; access via signed URL.	Only content needed for embedding/generation is sent.
Extracted text/chunks	PostgreSQL; workspace-scoped; restricted.	Embeddings and selected context are sent to configured provider.
Chat content	PostgreSQL message records.	User question and bounded context sent during generation.
Usage/audit events	PostgreSQL with minimal metadata.	Never sent.
Logs	Structured, sanitized, retention-limited.	Never sent.

Privacy requirements

- Publish a privacy notice that identifies providers, categories of data processed, purpose, retention and user deletion process.
- Use provider paid API tier for confidential user documents where contract/terms state submitted data is not used to improve provider products; re-check provider terms on launch.
- Never use user documents for training, evaluation datasets or analytics without an explicit separate opt-in and documented consent.
- Keep logs free of raw file contents, access tokens, passwords, signed URLs and full prompts by default.
- Provide account data export and account deletion workflow; set explicit retention windows for backups and deleted data.

Backup / recovery baseline

Asset	Backup approach	Recovery check
PostgreSQL	Nightly encrypted `pg_dump` to private R2 backup prefix; retain 7 daily + 4 weekly snapshots.	Monthly restore into isolated DB and run integrity checks.
R2 source files	Provider durability plus optional versioning/lifecycle; do not assume DB backup restores objects.	Sample object restore and checksum verify.
Secrets	Password manager / deployment secret store; never in backups repo.	Document rotation procedure.
Code/config	Git repository + tagged releases + infra files.	Rebuild from clean VPS using runbook.

Deletion semantics

On user request, revoke sessions and hide resources immediately. Queue deletion of conversations, messages, documents, chunks, summaries, R2 objects and derived artifacts. Keep only minimal tombstone/audit metadata where legally/operationally required. Delete backup copies according to documented expiry; disclose backup retention. Deletion jobs are idempotent, retryable and report completion to the user.

34 / Deployment architecture and Docker Compose

Production topology for the ₹500/month infrastructure target.

Costed baseline (verified planning snapshot)

Service	Current planning assumption	Monthly planning cost
OVHcloud VPS-1	2 vCores, 4 GB RAM, 40 GB NVMe; published starting price ₹420 before GST.	₹420 + 18% GST = ₹495.60
Cloudflare Pages	Static assets; Free plan limits include 500 builds/month and static asset requests are free/unlimited.	₹0 within plan
Cloudflare R2	10 GB-month Standard storage, 1M Class A and 10M Class B operations/month included; egress free.	₹0 within free allowance
Cloudflare DNS / SSL	Free DNS and Universal SSL; Full (strict) to origin.	₹0 within plan
Gemini API	Usage-based; not included in infrastructure estimate.	Variable
Domain / email	Domain renewal and transactional email provider.	Separate / variable

BUDGET TRUTH The VPS alone consumes almost the full ₹500 target after GST at the displayed starting price. This assumes the advertised commitment/region price is available at checkout. If checkout is higher, choose a lower-cost provider or raise the budget; do not pretend AI usage and domain are included.

VPS service layout

```
Docker Compose services:
  caddy     - reverse proxy, HTTPS, security headers
  api       - FastAPI app (2 workers initially only if memory allows; start 1)
  worker    - same image, COMMAND=worker, concurrency=1
  postgres  - PostgreSQL + pgvector, private volume/network

Cloudflare Pages hosts static Next.js export.
R2 stores files; Gemini APIs provide embeddings/generation.
```

Deployment requirements

- Use Ubuntu LTS on VPS; SSH key only, disable password login/root login after confirming break-glass access.
- Expose only 22 (restricted source IP where practical), 80 and 443; PostgreSQL has no host-published port.
- Use Docker Compose health checks, restart policies, resource limits and named persistent volumes.
- Use Cloudflare proxy with Full (strict) TLS; origin only accepts Cloudflare traffic where firewall/IP maintenance is practical.
- Set 1–2 GB swap only as emergency headroom; do not count swap as RAM for throughput.
- Keep source document bytes in R2, not on the VPS volume; temporary worker files are cleaned after processing.

35 / CI/CD, release, backup and recovery operations

Releases must be repeatable and reversible.

CI pipeline

On pull request:
1. lint + format check
2. Python type check + frontend type check
3. unit tests + integration tests with PostgreSQL/pgvector
4. dependency vulnerability scan
5. Docker build + smoke test
6. upload test results / build artifacts

On protected main tag:
1. build immutable backend image by commit SHA
2. deploy static frontend to Cloudflare Pages
3. pull backend image on VPS
4. run backup checkpoint + Alembic migration
5. restart API/worker with health check
6. run smoke tests; rollback image if unhealthy

Release rules

- Use protected main branch and required checks. Production secrets are GitHub environment secrets with approval if available.
- Use immutable Docker image tags (commit SHA); do not deploy `latest` as the only reference.
- Run backward-compatible expand/migrate/contract database changes. Take a fresh backup before risky schema changes.
- Frontend and API must remain compatible during staged deployment; use additive API fields before removing old ones.
- After deployment, verify `/health/live`, `/health/ready`, authentication, document listing and a synthetic AI request with a safe test document.
- Keep previous known-good image tag and documented rollback command. A code rollback does not reverse destructive migrations; plan separately.

Backup runbook

```
# Example operator workflow (adapt paths/secrets before execution)
docker compose exec -T postgres pg_dump -Fc -U docmind docmind > backup.dump
gzip -9 backup.dump
# Upload encrypted artifact to private R2 backup prefix.
# Verify size, checksum and restore in isolated staging periodically.
```

Recovery objectives

Metric	Initial target	Notes
RPO	24 hours	Nightly DB dump; improve with WAL archiving when business justifies it.
RTO	4 hours	Rebuild VPS/containers and restore DB/object configuration.
Restore drill	Monthly	Restore DB to isolated instance; inspect users/docs/jobs/chunks.
Backup retention	7 daily + 4 weekly	Encrypt, restrict credentials and periodically test retrieval.

36 / Observability, reliability and performance

Make failures diagnosable before real users depend on the service.

Health and monitoring

Signal	Implementation	Alert / operator action
Liveness	GET /health/live checks process event loop.	Restart only if process is dead.
Readiness	GET /health/ready checks DB and critical config.	Do not route traffic when not ready.
Worker heartbeat	Persist last heartbeat and current job.	Alert if worker stale or oldest queued job too old.
API latency	Structured request duration; p50/p95 by route.	Investigate slow DB/provider calls.
Errors	Sentry + sanitized logs + trace ID.	Group by error code/release.
Provider usage	Tokens, request count, 429/5xx, latency and estimated cost.	Alert at budget thresholds; kill switch if needed.
Storage	R2 bytes/operations and VPS disk usage.	Warn at 70%, critical at 85%.
DB	Connection count, slow queries, DB size, backup status.	Tune indexes or scale before saturation.

Reliability requirements

- Use bounded connection pools and provider timeouts. API must not hang indefinitely on a provider call.
- Use graceful shutdown: stop accepting new work, finish or release current job lease, close DB/provider clients.
- Apply retry only to idempotent transient errors; use jitter and caps to avoid retry storms.
- Use circuit breaker or temporary provider disable after repeated failure; expose degraded state to users.
- Run scheduled reconciliation for stale upload intents, orphan R2 objects, expired sessions and stale jobs.
- Keep database backups off-host; monitor disk and backup freshness.

Performance test targets (initial hypotheses)

For a small pilot, target API metadata endpoints p95 under 500 ms excluding network; document upload initiation under 500 ms; retrieval under 2 seconds excluding embedding API latency; complete Q&A latency should be measured end-to-end and reported rather than promised. Load test with realistic 10–20 simultaneous active sessions and large document ingestion. Record memory/CPU per worker job and lower concurrency or page limits if the 4 GB host approaches memory pressure.

> **PERFORMANCE HONESTY** Do not advertise throughput, concurrency or uptime numbers until measured in the deployed region and under representative document sizes.

37 / Testing strategy and release acceptance

Tests must prove functionality, factual grounding and tenant isolation.

Test pyramid

Level	Coverage	Examples
Unit	Pure business rules and adapters.	Quota calculation, chunk boundaries, prompt assembly, citation parser, token budget.
Integration	Real PostgreSQL/pgvector + fake provider/storage.	Migrations, job claim, repository scoping, signed URL orchestration, usage events.
Contract	API request/response and error schemas.	OpenAPI validation, status codes, pagination, idempotency.
End-to-end	Browser flows against deployed-like stack.	Signup → verify → upload → ready → chat → citation → delete.
Security	Negative and abuse cases.	Cross-tenant IDOR, XSS, oversized files, archive bombs, auth replay, prompt injection.
AI evaluation	Golden question-answer dataset.	Recall@k, citation precision, answer support, no-answer correctness, latency and cost.

Minimum release acceptance checklist

- Fresh install from README succeeds with Docker Compose and `.env.example` values replaced.
- All migrations apply on an empty database and upgrade tests pass.
- Auth lifecycle and token rotation work; rate limits and generic password-reset response are tested.
- PDF, DOCX, TXT and MD fixtures reach READY and are searchable; malformed/password-protected/oversized fixtures fail safely.
- RAG answers include only validated citations from selected documents; no-evidence query returns explicit insufficiency.
- Cross-tenant access tests fail closed for documents, chunks, conversations, citations, signed URLs and jobs.
- Conversation, summary, extraction and comparison persist and re-authorize on every read.
- Budget limits, question quotas, output caps and provider kill switch block new AI calls as expected.
- Delete document/account hides resources immediately and cleanup retries until completion.
- Backups can be restored in an isolated environment; deployment rollback is documented and tested.
- Sentry/logs contain trace IDs but no secrets, signed URLs, raw document text or passwords.

AI evaluation dataset

Create at least 50 manually reviewed question/answer cases across 10–15 varied documents: direct fact, multi-hop, comparison, exact phrase, ambiguous question, missing answer, contradictory sources, table-derived fact, scanned page and prompt-injection document. Track retrieval recall, citation precision, answer faithfulness and correct abstention. Keep this dataset sanitized and licensed for testing.

> **RELEASE GATE** Do not launch publicly if the cross-tenant isolation suite fails, backups have not been restored once, citations can point to unauthorized sources, or the spend cap can be bypassed.

38 / Roadmap, risks, configuration and references

Final execution plan and source-of-truth configuration.

Recommended delivery roadmap

Phase	Deliverables	Exit gate
Week 1: foundation	Repo, auth, workspace, DB migrations, Docker, static frontend skeleton.	Secure signup/login; deployment boots.
Week 2: documents	R2 upload, file validation, library, status model, worker queue.	Upload and delete lifecycle works.
Week 3: ingestion	PDF/DOCX/TXT/MD parsing, OCR fallback, chunks, embeddings, pgvector/FTS.	Documents searchable; parser tests pass.
Week 4: RAG	Hybrid retrieval, citations, chat history, no-answer handling.	Golden Q&A and isolation tests pass.
Week 5: productivity	Summary, key points, extraction, comparison, usage meter.	Actions persist; quota and budget tests pass.
Week 6: launch	Monitoring, backups, security review, CI/CD, beta onboarding.	Acceptance checklist complete; restore drill passes.

Top risks and mitigations

Risk	Mitigation
₹500 cap too low at checkout	Treat ₹500 as target, not guarantee; verify VPS region/term; keep fallback provider/host option.
4 GB RAM exhaustion	Static frontend, one worker, OCR limits, bounded API workers, monitor memory.
AI hallucinations	Source-only prompt, retrieval thresholds, citation validation and evaluation set.
Provider pricing/model change	Adapter + config + cost alert + model evaluation before switching.
Sensitive document exposure	Paid API terms, private storage, no training, least privilege, explicit retention notice.
Backup incomplete	Separate DB/object restore procedures and monthly drills.

Required environment variables

```
APP_ENV=production
APP_SECRET_KEY=<random-32+-byte-secret>
DATABASE_URL=postgresql+psycopg://...
PUBLIC_APP_URL=https://app.example.com
API_BASE_URL=https://api.example.com
CORS_ALLOWED_ORIGINS=https://app.example.com
R2_ACCOUNT_ID=... R2_BUCKET=docmind-private
R2_ACCESS_KEY_ID=... R2_SECRET_ACCESS_KEY=...
GOOGLE_API_KEY=...
LLM_PROVIDER=google LLM_MODEL=gemini-3.8-flash
EMBEDDING_PROVIDER=google EMBEDDING_MODEL=gemini-embedding-2
EMBEDDING_DIMENSION=768
AI_FEATURES_ENABLED=true AI_MONTHLY_BUDGET_USD=<cap>
MAX_FILE_MB=20 MAX_PAGES=200 WORKER_CONCURRENCY=1
```

Authoritative references (checked 01 Nov 2026)

OVHcloud India VPS pricing — https://www.ovhcloud.com/en-in/vps/ Published VPS-1 starting price/specification; verify region, commitment and final checkout.

Cloudflare R2 pricing — https://developers.cloudflare.com/r2/pricing/ Storage, operation allowances, free tier and egress fees.

Cloudflare Pages limits — https://developers.cloudflare.com/pages/platform/limits/ Free-plan build and asset limits.

Cloudflare Pages pricing — https://developers.cloudflare.com/pages/functions/pricing/ Static asset request pricing.

Gemini API pricing — https://ai.google.dev/gemini-api/docs/pricing/ Current model token prices and free/paid data-use notes.

Gemini embeddings — https://ai.google.dev/gemini-api/docs/embeddings Embedding model IDs, output dimensions and task details.

Supabase pricing (alternative only) — https://supabase.com/pricing Free-plan limits and project pausing; not selected for primary production DB.

Final agent instruction: implement this specification in small, testable milestones. Where a provider API changes, consult its current official documentation, update the adapter/configuration and record the change. Do not silently weaken security, citations, quotas, deletion or acceptance criteria to make a demo pass.